P2.10 — dashboard fixes before the API

Status: all five items done, 10 October 2026; item 5 with a known limitation (D-89) **Position:** after the Phase 2 checkpoint, before Phase 3

Why this exists as its own item

Phase 2 closed as met: the P2.9 gate passed 5 of 5 and the checkpoint condition — a deployed app that renders a real network and lets someone correct a grey node — is satisfied. Its definition of done is not reopened to absorb later work. A DoD amended after it is met records a phase that kept moving rather than one that finished.

Five dashboard faults were raised on 8 October. None touches the API or the MCP server, so none belongs in Phase 3; all five are on surfaces a first-time visitor uses in the first minute. Two further items from the same list — the retrieval cap and confidence ranking — do belong in Phase 3 and are handled in the P3.8 amendment, because the API and the MCP server expose the same retrieval and the answer shape should be decided once.

P2.10 is dashboard only. Nothing in it changes what the classifier does, what is sent to any model, or what any figure means.

The five items

P2.10.1 — Scroll-to-zoom, scoped to the canvas (done)

Scrolling does not zoom the 3D view at all. That is deliberate: P2.8c (`7923f88`) set `scrollZoom` off because the plot swallowed the page scroll, so reaching anything below it meant fighting the canvas. This item reverses that decision, so it has to answer the reason it was made. Re-enable it with the scoping that makes it safe: the wheel zooms only while the pointer is over the canvas, and scrolls the page everywhere else.

Both halves matter. A canvas that captures the wheel wherever the pointer sits traps a visitor who is trying to read the page; one that never zooms makes the network unreadable past about a hundred nodes.

Done when: the wheel zooms with the pointer over the canvas, scrolls the page with the pointer outside it, and neither case scrolls the other.

P2.10.2 — The canvas holds its place (done)

Scrolling the left or right panel moves the centre panel out of view, leaving whitespace where the network was. The canvas should stay put while the side panels scroll.

Done when: scrolling either side panel to its full extent leaves the canvas fully visible and in position, at the layout widths the app ships with.

Result, items 1 and 2. These were browser verdicts, run on the deployable build. A first attempt (`50af986`) pinned the canvas with sticky CSS in a keyed container. In the browser the wheel over the chat carried the network with it, and a stale canvas from an earlier run was left on the page (D-85). That attempt was abandoned rather than repaired. What ships (`a0ac6bf`): - a 2:1 canvas-to-chat split - the chat and the review queue in a fixed-height panel the canvas's height, with its own scroll, so the page no longer grows below the canvas - "Your profile" under the canvas, within its column

#	Check	Verdict
1	Wheel over the canvas zooms; the page does not move	**Pass.** P2.8c's original fault, the plot swallowing the page scroll, did not return
2	Wheel over the chat panel scrolls only the panel, including past its end	**Accepted, not a defect.** Once the queue scrolls out, the page shifts by the margin above and below the columns (the counter, the provenance line, the attribution). The canvas does not scroll away
3	The chat panel scrolled to the end of the queue leaves the canvas in place	**Pass**
4	Scrolling the sidebar leaves the canvas in place	**Pass**
5	"Your profile" sits under the canvas, at its width	**Pass**
6	D-85's sequence leaves no stale canvas	**Pass, three runs**, each in a fresh private window, on a 60-person synthetic file. D-85 is closed on the mechanism, with its cause unidentified. A run on the 442-person synthetic export is part of the live check after the next cut
7	The pager and the skip caption are reachable inside the panel	**Pass**

Check 2's shift has a known fix, `overscroll-behavior: contain` on the panel. It is deferred, because its clean selector is a keyed container, which is one of D-85's two unresolved suspects.

P2.10.3 — Newest chat answer first (done)

New questions and their answers are appended below the history, so the thing just asked is the thing furthest from the input. Reverse it: most recent at the top, descending.

Done when: after three questions the most recent exchange is at the top, and the order survives a rerun. Within an exchange, the question stays above its answer; only the exchanges are reversed.

Why this order. Since P2.10.2 the chat history sits in a region of fixed height with its own scroll. Newest-first puts the newest answer in view without scrolling. Appended at the bottom, it would sit below the fold of a region that does not grow.

Layout. Storage stays chronological, and the renderer reverses a copy at paint time. From the top: the question box, then the panel's controls ("What is sent", "Clear conversation"), then the history. The box and the controls sit outside the scroll region, and a new question renders as the history's top entry. Also done when: asking a question while the history is scrolled partway down leaves the new answer visible without scrolling up. That last criterion is a browser verdict, and the one most at risk. Browsers keep the current view steady when content is added above it, which would leave the new answer out of view.

P2.10.4 — The chat input wraps (done)

The question box is one line, so a long question scrolls out of view as it is typed and cannot be read back before sending. It should wrap to a new line.

Done when: a question longer than the box width wraps, so it is visible without horizontal scrolling; four lines show a typical question whole; and an explicit Send button submits the question. A text area does not submit on Enter, so without a button the chat inherits D-51's failure on the notes box.

This criterion first said "the box grows rather than the text disappearing". Streamlit 1.41's text area has a fixed height and does not grow with its content. A question longer than four lines therefore scrolls inside the box rather than sideways. The text stays visible and can be read back, but the box does not grow. The fixed height is a property of the pinned version: the runtime lock has held `streamlit==1.41.1` since D-31, and `streamlit-keyup` is kept at 0.4.0 to match that pin, not the other way round. Revisit this if the pin moves.

Ctrl+Enter also sends, as it does in the notes box. A text area in a form submits on Ctrl+Enter. D-51 did not rule that out. It ruled out Ctrl+Enter or leaving the box being the only, invisible way to keep a note, and fixed it with a visible Save button in a form, which still accepts Ctrl+Enter. The chat now matches it: Send is the primary path, and Ctrl+Enter is a convenience beside it. The two boxes behave alike.

Result, items 3 and 4. These were browser verdicts on `b00e84e`.

#	Check	Item	Verdict
1	A new question's answer lands at the top, above the previous exchange, and stays there after a rerun	3	**Pass**
2	A long question wraps, with no sideways scrolling	4	**Pass**
3	Enter starts a new line and sends nothing	4	**Pass**
4	Leaving the box sends nothing, and the text is kept	4	**Pass**
5	Send sends once, and the box empties	4	**Pass**
6	No answer from the answering run lingers, checked twice with a long answer	3	**Pass**
7	Ctrl+Enter sends, as it does in the notes box	4	**Pass**
8	Asking with the history scrolled partway down leaves the new answer visible	3	**Fail.** The answer was out of view, above, and needed scrolling up to see
9	The region's bottom edge against the canvas's, at film width and at 1280 px	3	**Not measured**
10	After three questions the queue is still findable below the history	3	**Pass**

Item 4 is done. Item 3 stays open on verdict 8, and verdict 9 still needs its number. Verdict 8 failed the way scroll anchoring predicts. The history region is reused across reruns, so its scroll position survives, and the browser keeps the current view steady while the new exchange is inserted above it. No remedy is applied yet.

The remedy chosen for verdict 8: the question box moves to the top of the scroll region. The panel's controls move in with it, so they stay directly under the box. Only the chat's heading and coverage line sit above the region. Typing now means being at the top of the region, so a new answer lands under the box, in view by construction rather than by browser behaviour. This withdraws the layout's "box outside the scroll" requirement: the box scrolls away while older answers are read. Two alternatives were set aside: - A fresh history region for each exchange would need a keyed container, which is D-85's open suspect. - Shifting the region's position to defeat reuse would deliberately create the moving element D-85 suspects.

Verdicts 8 and 9 are to be re-run on this layout.

Height. The canvas grows from 620 to 760 px, and the chat region follows it at 670 px (the canvas height less an estimated 90 px for the heading and the coverage line). The region was small next to a sidebar that runs to the foot of the window. A region filling the window like the sidebar would need CSS, whose clean selector is a keyed container, D-85's open suspect. A region taller than the canvas would grow the page below the canvas and partly undo P2.10.2. Raising both together keeps the two columns ending at the same place. 760 px is sized for a 1080-pixel-tall screen; a smaller screen scrolls the page a little.

Re-run on `e1625d6`:

#	Check	Verdict
8	Asking after scrolling the history down and back up to the box leaves the new answer visible	**Pass**
9	The chat region's bottom edge against the canvas's	**Pass, judged by eye** at the author's screen size: the two columns end together. Not measured in pixels, and not checked at a second width

Item 3 is done. If the bottom edges part at the recording's screen size, the fix is `CHAT_TOP_ESTIMATE` in `app.py`.

Verdict 9 measured, 10 October. In the browser, with a snippet reading the bounding boxes of the chat region and the canvas. The layout is the one judged above: the only change to `app.py` since `e1625d6` is the timing wrapper around the same calls. Delta is the chat region's bottom less the canvas's. Positive means the chat column ends below the canvas.

Window	Delta	Note
1920 px	**+28 px**, as printed	Region bottom 1127, canvas bottom 1098, both as displayed. The snippet rounds the difference of the unrounded values; subtracting the displayed figures gives 29
1365 px, DevTools docked	**+84 px**	A 1365 px viewport inside a 1920 px window, not a 1365 px browser. Indicative of the narrow end, not a substitute for 1280
1280 px	**Not measured**	

The overshoot grows as the window narrows, because the captions above the region wrap and push it down. One constant can't fit a stack whose height changes with width. The overshoot is cosmetic: the chat column ends a little below the canvas, and nothing is clipped or scrolls wrong. **`CHAT_TOP_ESTIMATE` stays at 90.** A

negative delta would be different: the chat column ending above the canvas leaves empty space under it, the failure visible at a glance. Whoever measures 1280 should look for that.

P2.10.5 — Chat history in a saved session (done, with D-89)

A downloaded session does not carry the chat. Restoring it loses every question and answer from that session.

This changes the session file format to **version 3**. Version 2 shipped on 8 October (D-84), so this is the second format change in as many days, and the rules it established hold:

- a version-2 file restores into a version-3 session, with an empty chat history
- an older build refuses a version-3 file through the existing "saved by a newer version" path
- the chat history restores in the order P2.10.3 defines, not the order it was stored in

Done when: a session downloaded after three questions restores with those three exchanges, in order; a version-2 file restores without error and without a phantom chat; and an older build refuses a version-3 file rather than reading it partially.

Built (`824644f`). - The version-2 restore test was committed first (`5b8c3de`), before the writer existed. It restores a literal version-2 file through `_apply_upload` and checks the corrections, notes, profile text and reading arrive intact. - The file is version 3, and carries `"chat"` in chronological order, as the session stores it. The renderer reverses it for display, as it does live. - A restored chat replaces the session's. A version-1 or version-2 file restores an empty one, and malformed entries are dropped. - The download's privacy line now says the file holds the chat, whose answers name connections.

Tests, one per mutation: - the chat saved in display order - the version not bumped - a version-2 file producing a phantom exchange - a restored chat shown in storage order - an older build's refusal not firing

Each mutation fails at least one test.

Result, 10 October. Browser verdicts. Verdicts 1 and 2 were run on the app at HEAD (`0fe5ca2`, the same session code as `824644f`). Verdict 3 was run on the version-2 build (`5b8c3de`), served from a separate worktree so main was untouched.

#	Check	Verdict
1	Three questions, download, Start over, restore: three exchanges, newest first, each question above its answer	**Pass.** The file is version 3, and its chat holds the six turns in chronological order
2	A version-2 file restores into this build: the 4 corrections, the note, the profile title and About text, and the reading; an empty chat, with no phantom exchange; no highlight left on the canvas	**Pass,** on the second version-2 file (see below)
3	The version-2 build refuses a version-3 file, and nothing in the session changes	**The refusal works; its message is lost (D-89).** No "saved by a newer version" message appeared. An earlier restore's "Re-applied 4 of 4 corrections and 1 of 1 note." stayed beside the refused file

Why verdict 2 needed a second file. The first version-2 file was saved with corrections, a note and profile text, but without a reading. The reading (`enrichment.result`) is the field D-84 changed, from a Comparison to one Classification, and the reason the format is version 2. Restoring that file would have

passed while testing only the fields versions 1 and 2 share. Its keys were read before the verdict ran, and the file was saved again with a reading. The README's "Tests and verdicts" rule comes from this.

What verdict 3 shows, and what it can't. The older build does refuse. The leftover "4 of 4" report proves it: the version-3 file holds 0 corrections and 0 notes, so a read would have replaced that report. The refusal message is drawn and then thrown away by the `st.rerun()` that follows every upload. That is D-89, and it has been in the live app since September. The check as specified couldn't have discriminated on its own: the version-3 file carries no corrections, notes or profile text, so "nothing in the session changed" would also be true of a partial read. D-89 is fixed after the cut, as the next item.

The item's three done-when clauses: the chat restores in order (met); a version-2 file restores without error and without a phantom chat (met); an older build refuses a version-3 file rather than reading it partially (met in what it does, but the refusal is invisible, which is D-89).

What P2.10 is not

- **Not a chat feature change.** The retrieval cap, confidence filtering and per-row rationale are Phase 3 (P3.8 amendment). P2.10 does not change which people an answer returns or how an answer is worded.
- **Not a canvas rewrite.** Items 1 and 2 are the wheel's scope and the panel's position. Nothing about the projection, the layout or the markers changes.
- **Not a fix for D-72.** The build still has no progress indicator. That is its own entry and is unaffected by this work.

Order

1 and 2 together — both are canvas and layout. 3 and 4 together — both are the chat panel. 5 last, because it depends on 3's ordering being settled.

Risks worth naming

Item 5 is the only one that can lose data. A format change that drops a restored session's corrections or notes would be worse than the fault it fixes. The version-2 restore path gets a test before the version-3 writer does.

Item 4 can lose questions. A wrapping box is a text area, and a Streamlit text area submits on Ctrl+Enter or when focus leaves it, not on Enter. That was D-51's failure on the notes box, which kept a note only on Ctrl+Enter or on leaving the box until an explicit Save button replaced it. A wrapping chat input needs an explicit Send control for the same reason.

Items 1 and 2 are the hardest to test automatically. Wheel scoping and sticky positioning are browser behaviour, not Python. Each needs a written verdict from a browser session, recorded the way the P2.9 gate steps were, rather than a passing unit test standing in for a thing nobody looked at.